

SF BAY AREA LINUX KERNEL & eBPF MEETUP

Journey of AixKer

A kernel-level load balancer in Rust and XDP —
and the packets that kept disappearing

Ali Heidari

aixker.com

Every load balancer runs in userspace

1

Packet hits
the NIC

2

Driver
allocates an
sk_buff

3

Full network
stack
traversal

4

Copied to
userspace

5

Now we pick a
backend

We pay the full cost of the kernel network path just to answer one routing question.
And the thing answering it is a single point of failure.

What if the decision happened at the NIC — and every node made it for itself?

EARLIEST POINT ON THE MACHINE

XDP — before the `sk_buff` exists

HOW A NODE KNOWS IT IS FULL

A small RL model, per node

One agent per node. No proxy tier.

USER SPACE — Rust, tokio

Holds the node's state, knows the peers, runs the model, pings the target

KERNEL SPACE — aya, XDP

Reads the maps, rewrites the packet, passes it up or redirects it out

The two halves never call each other. BPF maps are the API between them.

Five maps

MAP	WHAT IT HOLDS
SERVERMAP	One entry, 20 bytes. Empty means pass; populated means redirect.
NAT_MAP	Per-connection 5-tuple state. The one that took a week.
DEVMAP	Interface index for XDP_REDIRECT.
BLOCKLIST	Source IPs we drop.
PORT_COUNTER	Per-CPU ephemeral port allocation. No locks on the hot path.

Two states, and that is the whole policy

FREE

SERVERMAP is empty. Every packet passes up the stack. The local service handles it.

BUSY

The model says the node is loaded. Find a free peer, write it into SERVERMAP. New connections leave at the NIC.

Established connections are not affected by the switch. That took a while — slide after next.

I built gossip over QUIC. Then I deleted it.

QUIC

A connection to every peer. Setup, TLS, file descriptors — to say one short sentence.

UDP MULTICAST

One datagram. Everybody on the group hears it. The interface delivers it anyway.

Hello

IP, state, MAC — on startup and on every state change

Batch

Everything I know, in one datagram

Eviction

Forget this peer

No delivery guarantee — and the protocol never assumes one. Ten second TTL, reaper every five. Plus a one second health check on the peer that actually matters.

TWO CONTAINERS: WORKED. THREE CONTAINERS:

Some packets just disappeared

No error. No log line. And losing one packet doesn't degrade an HTTP request — it kills it.

BUG ONE

I was deciding per packet, not per connection

With two nodes there is one possible target, so every packet of a connection landed in the same place by accident. With three, packets of one HTTP request fanned out across peers. Each peer saw a fragment of a conversation it never agreed to.

KEY

src IP, dst IP, src port, dst port, protocol

VALUE

The rewrite this connection is under — both directions, compact enough to live in a kernel map

BUG TWO

The driver was eating them, and saying nothing

Change one bit of a packet and the checksum is invalid. The driver drops it brutally. No error, no log.

I was rewriting IPs, MACs and ports on every redirected packet — so every one of them was invalid.

aya's checksum helpers work on sk_buff. I'm in XDP — the sk_buff doesn't exist yet.

So: incremental delta, by hand, on the raw bytes. IP header and TCP header.

The idea took an afternoon. The checksum took a week.

Three things that came out of failures

Connections outlive the decision

Local IP and MAC are stored in the NAT entry at SYN time. An established flow carries its own rewrite state and stops caring what SERVERMAP says.

Failover in about 50ms

Ping the target every second. On failure, clear SERVERMAP, then ping every cached free peer at once. First one that answers wins.

Ports without locks

XDP runs on many CPUs at once. The ephemeral range is sliced per CPU, each cycling its own block. No contention, no collisions.

The model is smaller than people expect

It is not routing packets. It answers one question, on an interval: **given this machine's metrics, should this node take new work?**

WHAT IT IS

Two layers, ReLU and softmax. REINFORCE, replay buffer, epsilon-greedy on top of softmax sampling. Written from scratch so it stays light enough for every node.

HOW IT PLUGS IN

Two callbacks — one supplies the features, one computes the reward. Inference, training, or training on an interval while serving.

Honestly: I can show you it adapts. I cannot yet show you a controlled study proving it beats a well-tuned static threshold.

I would rather tell you these than have you find them

No TLS, no L7. This is layer 4. nginx or envoy at the edge, AlxKer behind it.

Gossip is unauthenticated. Any host on the subnet can claim to exist and be free.

NAT entries leak. Removed on reset, but a normal FIN teardown leaves them behind.

Needs privileges. CAP_BPF, CAP_NET_ADMIN, CAP_SYS_ADMIN.

A one second recovery window. Peers can hold a stale target until the next health check. Fix: invalidate on the state-change broadcast.

Multicast rules out most public cloud. Bare metal and on-prem work. A VPC needs a unicast fallback I haven't written.

Please try to break it

It is an MVP. It works, it has real gaps, and everything is on GitHub — the agent, the transport library, the RL trainer and its CLI.

SITE

aixker.com

GITHUB — REPOS STILL NAMED AION

github.com/ali-heidari

EMAIL

ali-heidari@outlook.com